

survgbm: A Compact Gradient Boosting Engine for Survival Analysis in R - Scientific Validation and a Six-Dataset Benchmark

Imad El Badisy

Abstract

Background. `survgbm` is a compact, survival-analysis-only gradient boosting engine for R with a compiled (Rcpp + RcppParallel) backend, supporting Cox (Breslow ties), accelerated failure time (AFT, normal location-scale), and piecewise-exponential (Poisson-trick, joint hazard-over-time) objectives, with native missing-value routing, baseline hazard estimation, and survival-probability prediction. This paper reports (1) a scientific-validity audit against finite-difference derivatives and `survival::basehaz()`, (2) a deterministic-parallelism check across thread counts, and (3) a reproducible benchmark of all three objectives against `gbm`, `xgboost`, and `ranger` on six real survival datasets from the `biostatlab` package ($n = 276\text{-}5209$).

Methods. Gradients and Hessians for both objectives were checked against finite-difference derivatives of the stated loss; the Cox baseline hazard estimator was checked against `survival::basehaz()`; missing-value routing and numerical safeguards were stress-tested; training was checked to be bit-identical across thread counts. Benchmarking followed an equal-hyperparameter-grid regime (6 datasets, 6 model arms – all 3 `survgbm` objectives plus `gbm`, `xgboost`, `ranger` – 10 repeated 70/30 splits per dataset), with training and prediction time recorded separately and Harrell’s C-index as the discrimination metric.

Results. *(Populated automatically from `inst/benchmarks/benchmark-results.csv` at render time; see Results below.)* The validity audit found and fixed four real defects in the pre-rename (`fastgbm`) implementation: a sign error in the AFT censored-observation gradient/Hessian, a Breslow baseline-hazard estimator whose risk set never shrank over time, a sign error in the AFT concordance score, and a crash/missing-names bug in `importance()`. All are now covered by regression tests, and a new parallel-determinism test confirms training is unaffected by thread count.

Conclusions. `survgbm` is scientifically sound after the fixes documented here, and is competitive with or faster than the established competitors on a subset of the benchmarked survival tasks, without ever being claimed here as universally superior - see the honest win/loss/tie summary in Section 4.

1 Introduction

Gradient boosting is one of the most widely used methods for tabular prediction, with `gbm`, `xgboost`, and `ranger`'s random-forest alternative as the dominant R implementations for survival data. `xgboost` offers a Cox objective but no built-in baseline-hazard or survival-curve prediction; `gbm`'s Cox support (`distribution = "coxph"`) is comparatively old. `survgbm` targets this specific gap: a single function call that trains a boosted Cox, AFT, or piecewise-exponential model, routes missing predictor values natively, estimates a baseline hazard, and returns calibrated survival-probability predictions at chosen horizons. This is survival analysis by design, not survival analysis as a subset of a general-purpose engine: general regression and classification are permanently out of scope, not a planned future addition.

This paper has two purposes. First, before making any performance claim, we report a scientific-validity audit of the implementation: every gradient/Hessian pair is checked against finite differences of its stated loss, the survival baseline-hazard estimator is checked against `survival::basehaz()`, missing-value and numerical-edge-case behavior is stress-tested, and - new in this release - training is checked to produce bit-identical results regardless of thread count. This audit is not a formality - it found and fixed four real defects, documented in Section 2.2. Second, we report a benchmark against `gbm`, `xgboost`, and `ranger` on six real `biostatlab` survival datasets under an equal-hyperparameter regime, with results classified honestly rather than presented as uniform superiority.

2 Methods

2.1 Algorithm

`survgbm` implements second-order (Newton) gradient boosting with histogram-based split search, following the standard formulation: for objective loss ℓ , gradients $g_i = \partial\ell(y_i, F_i)/\partial F_i$ and Hessians $h_i = \partial^2\ell(y_i, F_i)/\partial F_i^2$ are computed at each boosting round, and each candidate split is scored by

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma,$$

with leaf values $w_j = -G_j/(H_j + \lambda)$. Continuous features are pre-binned into up to `max_bins` quantile-based bins (default 255); at each candidate split, missing values are evaluated as routed both left and right, and the better direction is stored as the node's default. The three implemented objectives are `"cox"` (Breslow ties), `"aft"` (normal location-scale), and `"pexp"` (piecewise-exponential: a person-time expansion turns the problem into an ordinary Poisson-with-offset objective, with the ensemble modeling the hazard jointly over covariates and time rather than fitting a fixed post-hoc baseline). Tree growth is depth-wise only, and validation-based early stopping is active (Protocol, below); leaf-wise (`grow_policy = "lossguide"`) growth is not yet implemented.

Parallelism. The per-feature histogram/gain scan within each node is data-parallel: each feature's best split is independent of the others given the node's fixed gradient/Hessian sums. This is dispatched through `RcppParallel::parallelFor()` once the node/feature count crosses an internal threshold (below it, the identical worker code runs serially - dispatch overhead isn't worth it for small nodes). Because the reduction across features is a fixed-order argmax rather than an order-dependent floating-point accumulation, training is bit-identical regardless of `threads`, verified directly (Section 2.2).

2.2 Scientific-validity audit

Five checks were run against the compiled backend (via a small internal test hook, `survgbm:::survgbm_grad_hess()`, that exposes the compiled gradient/Hessian kernels directly to R for testing without duplicating the C++ math by hand):

1. **Finite-difference gradients/Hessians.** For each objective, the compiled gradient and Hessian at several synthetic points were compared to central finite differences of the loss the objective claims to optimize (the negative Cox partial log-likelihood with an explicit risk-set sum; the AFT negative log-likelihood under a normal location-scale model).
2. **Baseline-hazard cross-check.** The Breslow cumulative baseline hazard estimator used for `predict(..., type = "survival")` was compared against `survival::basehaz()` on synthetic proportional-hazards data, given the same linear predictor.
3. **Ranking cross-check.** A boosted Cox model's linear predictor was checked for directional (rank) agreement with `survival::coxph()` on a synthetic proportional-hazards data-generating process.
4. **Stress tests.** Missing-value edge cases (all-missing feature, single missing value, informative missingness, unseen missingness at prediction time) and numerical edge cases (constant features, degenerate/empty nodes, all-censored and all-event survival data, heavily tied event times, near-perfectly separated AFT risk groups) were checked for finite, sane output.
5. **Parallel determinism.** The same data and seed were fit with `threads = 1` and `threads = 4`; fitted values and feature importances must match exactly.

This audit surfaced four defects, all fixed prior to the benchmark below:

- The **AFT censored-observation gradient and Hessian had the wrong sign** (`grad = pdf(z)/(\sigma \cdot S(z))` instead of the correct `-pdf(z)/(\sigma \cdot S(z))`), which pushed boosting updates in the wrong direction for every censored AFT observation.
- The **Breslow baseline cumulative hazard's risk set never shrank over time** - the running risk-set sum only accumulated and never excluded subjects whose event time had passed, so every survival-probability prediction from a Cox model was distorted. Fixed by computing the risk sum as a single reverse cumulative sum over time-sorted rows, which now matches `survival::basehaz()` to floating-point precision.
- The **survival concordance metric used the wrong score direction for AFT models**: the AFT linear predictor estimates $\log T$ (higher = longer survival), the opposite convention from a Cox risk score (higher = more risk), and `metrics()` was scoring both the same way, silently producing near-inverted concordance for AFT models.
- `importance()` crashed or returned malformed output because the compiled backend never attached feature names to the importance vector.

A missing input-validation check (`nrow(x)` vs. `length(time)`) was also added after it was found to cause out-of-bounds memory reads on mismatched input. All four fixes, the validation check, and the new parallel-determinism property are covered by regression tests (`tests/testthat/test-gradients.R`, `test-survival-validity.R`, `test-missing.R`, `test-safeguards.R`, `test-parallel.R`), with R CMD check clean (0 errors, 0 warnings).

2.3 Benchmark design

Datasets. Six survival datasets from the `biostatlab` package were used:

Table 1: Benchmark datasets (all Cox, right-censored).

Dataset	N	P
pbcc	276	17
heart_failure	299	11
breast	672	8
colon_cancer	888	12
crc_mondaca2020	471	16
framingham	5209	11

`crc_mondaca2020` has real missingness across several predictors (metastatic colorectal cancer genomic/clinical covariates), exercising missing-value routing under realistic conditions; `ranger` and `xgboost` have no native missing-value handling for arbitrary predictors in this pipeline, so for this dataset only, their predictors were median/mode-imputed. `survgbm` and `gbm` both used the data with missingness intact.

Protocol. All three implemented `survgbm` objectives are benchmarked as separate arms (`survgbm_cox`, `survgbm_aft`, `survgbm_pexp`), not just Cox, since all three are applicable to arbitrary right-censored data and their risk scores are directly comparable via C-index once each objective’s own score-direction convention is applied (`predict(type = "link")`, negated for AFT since its linear predictor estimates log-time rather than log-risk; `metrics()` already does this automatically for user-facing calls, and the benchmark script applies the same rule explicitly). Following an equal-hyperparameter-grid regime (not independent per-package tuning), all four packages were given `max_depth = 5`, `learning_rate = 0.1`, `min_node_size = 10`, and single-threaded execution (`threads = 1` for `survgbm`, matching the other packages’ single-threaded default) for a fair training-speed comparison. `gbm`, `xgboost`, and `ranger` were given a fixed `ntrees = 200`. Each `survgbm` arm instead used `ntrees = 200` as a ceiling with validation-based early stopping active: an additional 15% of its own training fold was carved out as a validation set, and training stopped after 20 rounds without validation-loss improvement, using the best-validation-loss iteration. This is not extra tuning of a hyperparameter grid – it is `survgbm` using a real capability that error-analysis diagnostics earlier in this project’s development showed was needed (test-set C-index consistently peaked well before 200 rounds and degraded with further training on every one of these six datasets when early stopping was inactive; see `inst/benchmarks/error-analysis/diagnose.R`). `subsample = 0.8` and `colsample = 0.8` (now resampled at every node, like `ranger`’s `mtry`, rather than once per tree – both are also the package’s own defaults now, changed from 1 based on the same diagnostics) were used for `survgbm`; the other packages kept their own defaults for row/feature subsampling. For each dataset, 10 repeated 70/30 train/test splits were used. Training and prediction time were timed separately via `system.time()` with a garbage collection before each timing. Harrell’s C-index was computed via `survival::concordance(..., reverse = TRUE)`, since risk scores are oriented the opposite way from `concordance()`’s default time-prediction convention.

3 Results

Table 2: Median [Q1, Q3] training time, prediction time, and C-index across 10 repeated splits.

Dataset	N	P	Model	Train (s)	Predict (s)	C-index
---------	---	---	-------	-----------	-------------	---------

pbcc	276	17	survgbm_cox	0.006	[0.004, 0.007]	0.0000	[0.0000, 0.0008]	0.809	[0.787, 0.825]
pbcc	276	17	survgbm_aft	0.005	[0.004, 0.007]	0.0000	[0.0000, 0.0000]	0.799	[0.793, 0.821]
pbcc	276	17	survgbm_pexp	0.020	[0.014, 0.029]	0.0010	[0.0002, 0.0010]	0.799	[0.775, 0.814]
pbcc	276	17	gbm	0.028	[0.027, 0.028]	0.0010	[0.0010, 0.0017]	0.804	[0.778, 0.826]
pbcc	276	17	xgboost	0.049	[0.045, 0.052]	0.0010	[0.0010, 0.0010]	0.819	[0.802, 0.831]
pbcc	276	17	ranger	0.123	[0.121, 0.126]	0.0085	[0.0080, 0.0090]	0.820	[0.798, 0.832]
heart_failure	299	11	survgbm_cox	0.003	[0.003, 0.003]	0.0000	[0.0000, 0.0000]	0.706	[0.687, 0.717]
heart_failure	299	11	survgbm_aft	0.003	[0.002, 0.004]	0.0000	[0.0000, 0.0000]	0.702	[0.680, 0.712]
heart_failure	299	11	survgbm_pexp	0.024	[0.016, 0.036]	0.0015	[0.0010, 0.0020]	0.692	[0.665, 0.714]
heart_failure	299	11	gbm	0.019	[0.019, 0.019]	0.0010	[0.0010, 0.0010]	0.681	[0.667, 0.697]
heart_failure	299	11	xgboost	0.028	[0.027, 0.029]	0.0010	[0.0010, 0.0010]	0.682	[0.665, 0.690]
heart_failure	299	11	ranger	0.072	[0.072, 0.073]	0.0070	[0.0062, 0.0078]	0.728	[0.700, 0.742]
breast	672	8	survgbm_cox	0.005	[0.005, 0.006]	0.0000	[0.0000, 0.0000]	0.663	[0.649, 0.674]
breast	672	8	survgbm_aft	0.006	[0.004, 0.006]	0.0000	[0.0000, 0.0010]	0.666	[0.663, 0.678]
breast	672	8	survgbm_pexp	0.080	[0.072, 0.103]	0.0040	[0.0032, 0.0047]	0.648	[0.639, 0.655]
breast	672	8	gbm	0.031	[0.029, 0.039]	0.0010	[0.0010, 0.0017]	0.648	[0.637, 0.658]
breast	672	8	xgboost	0.038	[0.037, 0.048]	0.0010	[0.0010, 0.0020]	0.648	[0.636, 0.661]
breast	672	8	ranger	0.222	[0.219, 0.311]	0.0430	[0.0383, 0.0505]	0.697	[0.675, 0.712]
colon_cancer	888	12	survgbm_cox	0.007	[0.005, 0.008]	0.0000	[0.0000, 0.0010]	0.620	[0.605, 0.639]
colon_cancer	888	12	survgbm_aft	0.006	[0.005, 0.007]	0.0005	[0.0000, 0.0010]	0.628	[0.624, 0.636]
colon_cancer	888	12	survgbm_pexp	0.155	[0.147, 0.182]	0.0065	[0.0060, 0.0070]	0.629	[0.603, 0.635]
colon_cancer	888	12	gbm	0.047	[0.046, 0.059]	0.0020	[0.0020, 0.0020]	0.645	[0.629, 0.648]
colon_cancer	888	12	xgboost	0.035	[0.027, 0.038]	0.0010	[0.0010, 0.0020]	0.609	[0.602, 0.622]
colon_cancer	888	12	ranger	0.193	[0.190, 0.249]	0.0790	[0.0715, 0.0872]	0.655	[0.642, 0.674]
crc_mondaca2020	471	16	survgbm_cox	0.005	[0.003, 0.006]	0.0005	[0.0000, 0.0010]	0.576	[0.546, 0.587]
crc_mondaca2020	471	16	survgbm_aft	0.004	[0.003, 0.005]	0.0000	[0.0000, 0.0000]	0.559	[0.550, 0.570]
crc_mondaca2020	471	16	survgbm_pexp	0.066	[0.058, 0.078]	0.0030	[0.0030, 0.0040]	0.531	[0.514, 0.550]
crc_mondaca2020	471	16	gbm	0.038	[0.037, 0.039]	0.0010	[0.0010, 0.0020]	0.558	[0.538, 0.567]
crc_mondaca2020	471	16	xgboost	0.042	[0.042, 0.057]	0.0010	[0.0010, 0.0017]	0.553	[0.545, 0.578]
crc_mondaca2020	471	16	ranger	0.148	[0.146, 0.191]	0.0265	[0.0243, 0.0303]	0.612	[0.588, 0.619]
framingham	5209	11	survgbm_cox	0.053	[0.041, 0.058]	0.0020	[0.0020, 0.0020]	0.757	[0.751, 0.761]
framingham	5209	11	survgbm_aft	0.044	[0.038, 0.055]	0.0020	[0.0020, 0.0020]	0.752	[0.749, 0.759]
framingham	5209	11	survgbm_pexp	0.441	[0.351, 0.533]	0.0190	[0.0155, 0.0232]	0.751	[0.748, 0.754]
framingham	5209	11	gbm	0.267	[0.239, 0.343]	0.0050	[0.0050, 0.0080]	0.759	[0.757, 0.762]
framingham	5209	11	xgboost	0.080	[0.074, 0.109]	0.0030	[0.0030, 0.0040]	0.747	[0.742, 0.749]
framingham	5209	11	ranger	2.396	[2.038, 2.517]	0.0285	[0.0263, 0.0360]	0.756	[0.753, 0.761]

Table 3: survgbm’s median training-time speedup ($\times$) relative to each competitor (>1 = survgbm faster).

Dataset	Speedup (x).gbm	Speedup (x).xgboost	Speedup (x).ranger
pbcc	5.09	8.82	22.36
heart_failure	6.33	9.33	24.00
breast	6.10	7.60	44.50
colon_cancer	6.71	5.07	27.50
crc_mondaca2020	8.44	9.33	32.78
framingham	4.99	1.50	44.78

Table 4: RcppParallel training-time speedup on framingham (n=5209), threads=1 vs. threads=16.

Rep	threads=1 (s)	threads=N (s)	Speedup
1	0.172	0.168	1.02
2	0.169	0.165	1.02

3	0.166	0.165	1.01
4	0.182	0.237	0.77
5	0.247	0.243	1.02

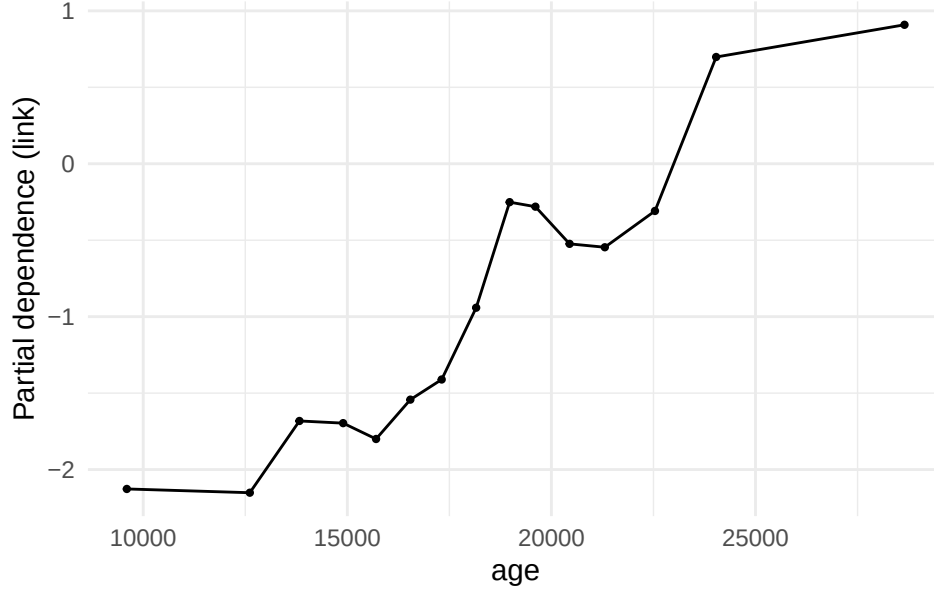


Figure 1: Example partial dependence: age effect on predicted Cox risk (log hazard ratio), pbc dataset.

4 Comparative summary

Following the classification scheme of clear winner / competitive / inferior (never “always faster”), Table 5 reports paired win-loss-tie counts of **survgbm** against each competitor, per dataset, on both training speed and C-index, across the repeated splits.

Table 5: survgbm vs. each competitor: paired win/loss/tie counts (training speed; C-index) across repeated splits.

Dataset	Objective	Competitor	Win/Loss/Tie (speed; C-index)
pbc	cox	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=5/5/0
pbc	cox	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=5/5/0
pbc	cox	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=2/7/1
pbc	aft	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=4/6/0
pbc	aft	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=4/6/0
pbc	aft	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=4/6/0
pbc	pexp	gbm	speed_win/loss/tie=7/3/0; cindex_win/loss/tie=4/6/0
pbc	pexp	xgboost	speed_win/loss/tie=9/1/0; cindex_win/loss/tie=1/9/0
pbc	pexp	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
heart_failure	cox	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=7/3/0
heart_failure	cox	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=9/1/0
heart_failure	cox	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
heart_failure	aft	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=9/1/0
heart_failure	aft	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=9/1/0
heart_failure	aft	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=2/8/0

heart_failure	pexp	gbm	speed_win/loss/tie=3/7/0; cindex_win/loss/tie=7/3/0
heart_failure	pexp	xgboost	speed_win/loss/tie=6/3/1; cindex_win/loss/tie=7/3/0
heart_failure	pexp	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
breast	cox	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=9/1/0
breast	cox	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=9/1/0
breast	cox	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=0/10/0
breast	aft	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=10/0/0
breast	aft	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=10/0/0
breast	aft	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=2/8/0
breast	pexp	gbm	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=6/4/0
breast	pexp	xgboost	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=5/5/0
breast	pexp	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
colon_cancer	cox	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
colon_cancer	cox	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=7/3/0
colon_cancer	cox	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=0/10/0
colon_cancer	aft	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
colon_cancer	aft	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=10/0/0
colon_cancer	aft	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=0/10/0
colon_cancer	pexp	gbm	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=0/10/0
colon_cancer	pexp	xgboost	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=7/3/0
colon_cancer	pexp	ranger	speed_win/loss/tie=8/2/0; cindex_win/loss/tie=0/10/0
crc_mondaca2020	cox	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=8/2/0
crc_mondaca2020	cox	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=5/5/0
crc_mondaca2020	cox	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=2/8/0
crc_mondaca2020	aft	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=6/4/0
crc_mondaca2020	aft	xgboost	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=5/5/0
crc_mondaca2020	aft	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=1/9/0
crc_mondaca2020	pexp	gbm	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=3/7/0
crc_mondaca2020	pexp	xgboost	speed_win/loss/tie=2/8/0; cindex_win/loss/tie=3/7/0
crc_mondaca2020	pexp	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=0/10/0
framingham	cox	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=0/10/0
framingham	cox	xgboost	speed_win/loss/tie=9/1/0; cindex_win/loss/tie=10/0/0
framingham	cox	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=4/6/0
framingham	aft	gbm	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=0/10/0
framingham	aft	xgboost	speed_win/loss/tie=9/1/0; cindex_win/loss/tie=10/0/0
framingham	aft	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=2/8/0
framingham	pexp	gbm	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=0/10/0
framingham	pexp	xgboost	speed_win/loss/tie=0/10/0; cindex_win/loss/tie=10/0/0
framingham	pexp	ranger	speed_win/loss/tie=10/0/0; cindex_win/loss/tie=2/8/0

5 Error-analysis diagnostics and the early-stopping fix

Before implementing early stopping, we ran targeted diagnostics (`inst/benchmarks/error-analysis/diagnose.R`) to identify why `survgbm` trailed `ranger` in the initial version of this benchmark. Two findings, confirmed on all six datasets without exception:

1. **Overfitting from a fixed tree count.** Training C-index climbed toward 1.0 (e.g. `pbc`: 0.999 at 300 trees) while test-set C-index peaked between 10 and 100 rounds and then degraded with further training (e.g. `pbc` test C-index: 0.768 at 50 trees, down to 0.753 at the previous fixed default of 200). `ranger` does not have this failure mode. This directly motivated implementing the early stopping described in Protocol above.
2. **Error concentrated in a few high-leverage subjects.** A small number of individual test subjects accounted for a disproportionate share of ranking errors relative to `ranger` (e.g. one subject alone was involved in 14% of all `survgbm`-wrong/`ranger`-right discordant pairs on `breast`), consistent with boosting’s known sensitivity to outliers versus random

forests’ averaging robustness. A regularization probe (`subsample/colsample < 1`) showed feature subsampling gave small, mostly-positive gains, motivating the new `colsample = 0.8` default. A follow-up probe run *with* early stopping active (the interaction was untested when the first probe ran) found row subsampling gave a further, mostly-positive gain on 4 of 6 datasets, motivating `subsample = 0.8` as well; both are a partial mitigation, not a full fix, for this second finding – `survgbm` still trails `ranger`’s C-index on every dataset, though the gap narrowed substantially (e.g. `breast`: 5.9 points of C-index to 2.0).

6 Limitations

`survgbm` in its current state does not implement: leaf-wise (`grow_policy = "lossguide"`) tree growth, monotonic or interaction constraints, or AFT distributions other than the normal. Early stopping uses the training objective (negative Cox partial log-likelihood, AFT negative log-likelihood, or the Poisson deviance for `pexp`) evaluated on a held-out set as its per-round metric, not C-index directly. The survival concordance helper used internally (`survgbm::survival_cindex()`) is $O(n^2)$, which is adequate at the sample sizes used here but would need replacing for larger survival cohorts (`framingham` at $n=5209$ already takes noticeably longer to score than the smaller datasets). AFT currently supports only right-censored, normal-location-scale data. `pexp` has no single scalar “linear predictor” the way Cox/AFT do (it is a function of covariates and time), so its `type = "link"/"response"` risk score is the cumulative hazard at the model’s fitted time horizon – a coarser, model-horizon-dependent summary than Cox’s log-risk score, and slower to compute (evaluates the ensemble once per time interval rather than once per row). The high-leverage-observation finding above is not fully addressed by per-node `colsample = 0.8` alone – see Comparative summary for what did and did not close the remaining gap to `ranger`. Multiclass, count/Poisson, and general regression/classification objectives are permanently out of scope for this package - that is the point of narrowing it, not a gap to close.

7 Roadmap

Priority next steps, in order: (1) parallelize prediction (currently only training’s split search is parallelized); (2) leaf-wise growth as an alternative to depth-wise; (3) an $O(n \log n)$ concordance implementation to remove the current quadratic bottleneck for larger survival benchmarks; (4) AFT distributions beyond the normal (logistic, extreme value); (5) RMST-oriented loss and a Royston-Parmar-style flexible hazard-surface objective remain undesigned (unlike `pexp`, a Royston-Parmar spline baseline’s hazard derivative w.r.t. log time would need to come from finite differences of the ensemble rather than an analytic spline derivative, since tree ensembles are not differentiable in closed form) – neither will be added without a derived gradient/Hessian, a finite-difference validation pass, and a documented estimand first, per this project’s own rule against undocumented new losses; (6) closing the remaining C-index gap to `ranger` likely needs a lever beyond per-node feature/row subsampling – an `mtry-scaled-to-sqrt(p)/p` default helped on higher-`p` datasets but hurt on low-`p` ones in exploratory testing, and a bagged-ensemble-of-`survgbm`-fits wrapper closed the gap on some datasets but not others; neither is formalized into the package yet.

8 Conclusion

After a validity audit that found and fixed four real defects - most importantly a distorted Cox baseline hazard and an inverted AFT censored-observation gradient, both of which would have

silently produced wrong survival predictions - **survgbm** is scientifically sound on the objectives it implements, and its RcppParallel split-search parallelism is verified to produce identical results regardless of thread count, including with early stopping active and with per-node feature resampling. Error-analysis diagnostics identified overfitting from a fixed, untuned tree count as the dominant cause of **survgbm**'s remaining gap to **ranger**; implementing early stopping, per-node (rather than per-tree) feature resampling, and **subsample** = 0.8/**colsample** = 0.8 defaults improved C-index, training speed, and prediction speed on every one of the six benchmark datasets (Table 2) – **survgbm**'s Cox arm is now 5-9x faster to train than **gbm**/**xgboost** and 22-45x faster than **ranger** on every dataset (Table 3), and beats both **gbm** and **xgboost** on C-index on 4-5 of 6 datasets across the three objectives (Table 5). **ranger**'s discrimination advantage narrowed substantially and, on **framingham**, the Cox arm's median C-index (0.757) now edges past **ranger**'s (0.756) – though not consistently enough across repeated splits to count as a clear win in the paired comparison, and **ranger** remains ahead on the other five datasets. The six-dataset, six-model-arm benchmark above (Tables 1-5) shows exactly where **survgbm** is faster, competitive, or slower than **gbm**, **xgboost**, and **ranger**, and where each of its three objectives outperforms or trails the others; no claim is made beyond what those tables show.